



Operating Systems

Lecturer

Dr. Fatma Ahmed HOSSAM El-Din



Chapter 5: CPU Scheduling

Chapter 5: CPU Scheduling

- Basic Concepts
- Scheduling Criteria
- Scheduling Algorithms
- Thread Scheduling
- Multiple-Processor Scheduling
- Real-Time CPU Scheduling
- Operating Systems Examples
- Algorithm Evaluation

Objectives

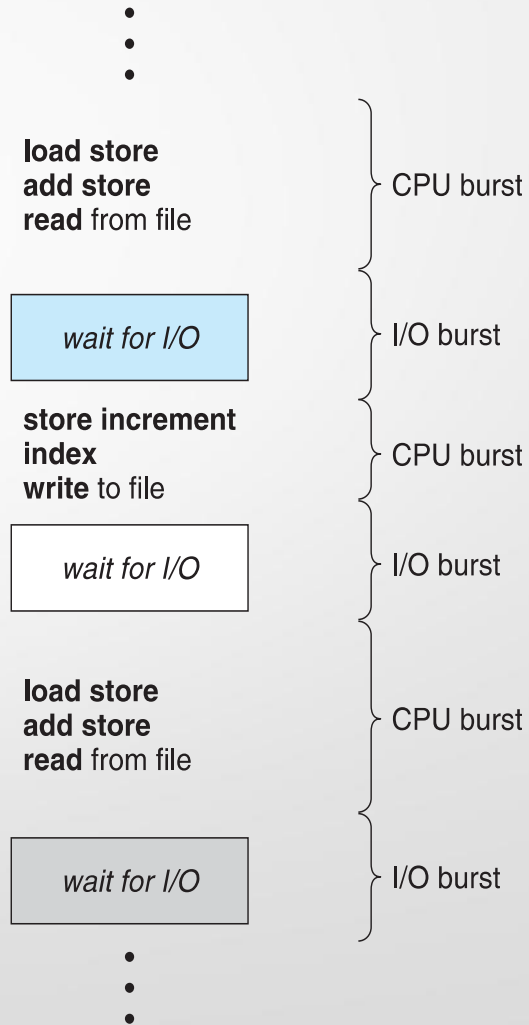
- To introduce CPU scheduling, which is the basis for multi-programmed operating systems
- To describe various CPU-scheduling algorithms
- To discuss evaluation criteria for selecting a CPU-scheduling algorithm for a particular system
- To examine the scheduling algorithms of several operating systems

Basic Concepts

- CPU Scheduling is the central in multi-programming system
- Maximum CPU utilization obtained with multiprogramming (Prevent CPU from being idle)
- Processes residing in main memory is selected by the scheduler that is:
 - Concerned with deciding a policy about which process is selected
 - Process selection based on a scheduling algorithm

Basic Concepts

- CPU–I/O Burst Cycle – Process execution consists of a **cycle** of CPU execution and I/O wait
- Processes alternate between these two states. Process execution begins with a **CPU burst**, Which is followed by an **I/O burst**, Which is followed by another CPU burst, then another I/O burst, and so on
- CPU bursts vary greatly from process to process and from computer to computer



CPU Scheduler

- Long-term scheduler choose some of them to go to memory (ready queue)
- Then, short-term scheduler (or CPU scheduler) chooses from ready queue a job to run on CPU bursts vary greatly from process to process and from computer to computer
- Medium-term scheduler may move (swap) some partially-executed jobs from memory to disk (to enhance performance)

CPU Scheduler

- Whenever the CPU becomes idle, the operating system must select one of the processes in the ready queue to be executed. The selection process is carried out by the :
- **Short-term scheduler** selects from among the processes in ready queue, and allocates the CPU to one of them
 - Queue may be ordered in various ways

CPU Scheduler

□ CPU scheduling decisions may take place when a process:

1. Switches from running to waiting state
2. Switches from running to ready state
3. Switches from waiting to ready
4. Terminates

CPU Scheduler

Scheduling can be:

- Non-preemptive

- Once a process is allocated the CPU, it does not leave until terminated

- Preemptive

- OS can force Preempt from a process from CPU at anytime
 - Say, to allocate CPU to another higher-priority process

CPU Scheduler

Non-preemptive and Preemptive

- Which is Harder to implement? And Why?

CPU Scheduler

Non-preemptive and Preemptive

- Which is Harder to implement? And Why?
- **Preemptive is harder:** Need to maintain consistency of data shared between process, and more importantly, Kernel data structure like I/O queues

Dispatcher

- Dispatcher module gives control of the CPU to the process selected by the short-term scheduler; this involves:
 - switching context
 - switching to user mode
 - jumping to the proper location in the user program to restart that program
- **Dispatch latency** – time it takes for the dispatcher to stop one process and start another running

Scheduling Criteria

- **CPU utilization** – keep the CPU as busy as possible
- **Throughput** – # of processes that complete their execution per time unit
- **Turnaround time** – amount of time to execute a particular process
- **Waiting time** – amount of time a process has been waiting in the ready queue
- **Response time** – amount of time it takes from when a request was submitted until the first response is produced, not output (for time-sharing environment)

Scheduling Algorithm Optimization Criteria

- **Maximize CPU utilization**
- **Maximize throughput**
- **Minimize turnaround time**
- **Minimize waiting time**
- **Minimize response time**

Scheduling Algorithms

There are many different CPU-Scheduling algorithm

1. First Come, First Served (FCFS).
2. Shortest job First (SJF).
 - Preemptive SJF
 - Non-Preemptive SJF
3. Priority
4. Round Robin
5. Multilevel Queues

First- Come, First-Served (FCFS) Scheduling

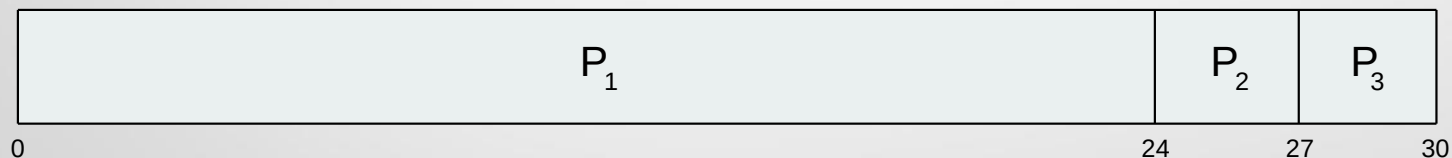
<u>Process</u>	<u>Burst Time</u>
----------------	-------------------

P_1	24
-------	----

P_2	3
-------	---

P_3	3
-------	---

- Suppose that the processes arrive in the order: P_1, P_2, P_3
The **Gantt Chart** for the schedule is:



- Waiting time for $P_1 = 0$; $P_2 = 24$; $P_3 = 27$
- Average waiting time: $(0 + 24 + 27)/3 = 17$

Waiting time		
P_1	P_2	P_3
0	24	27

First-Come, First-Served (FCFS) Scheduling

Suppose that the processes arrive in the order: P_1, P_2, P_3
The **Gantt Chart** for the schedule is:



- Waiting time for $P_1 = 0$; $P_2 = 24$; $P_3 = 27$
- Average waiting time: $(0 + 24 + 27)/3 = 17$

Waiting time		
P_1	P_2	P_3
0	24	27

Turnaround time		
P_1	P_2	P_3
24	27	30

Response time		
P_1	P_2	P_3
0	24	27

- Average Turnaround time: $(24 + 27 + 30)/3 = 27$
- Average waiting time: $(0 + 24 + 27)/3 = 17$

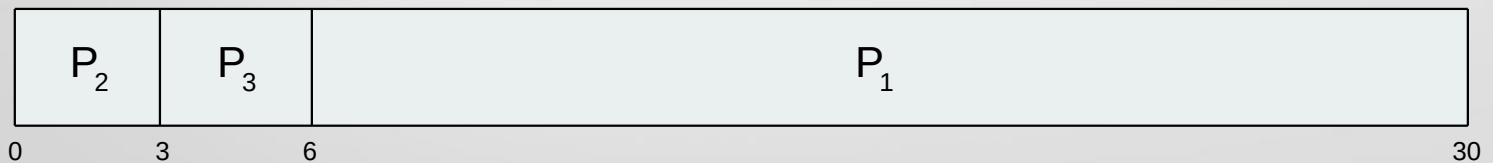
FCFS Scheduling (Cont.)

<u>Process</u>	<u>Burst Time</u>
P_1	24
P_2	3
P_3	3

Suppose that the processes arrive in the order:

$$P_2, P_3, P_1$$

- The Gantt chart for the schedule is:

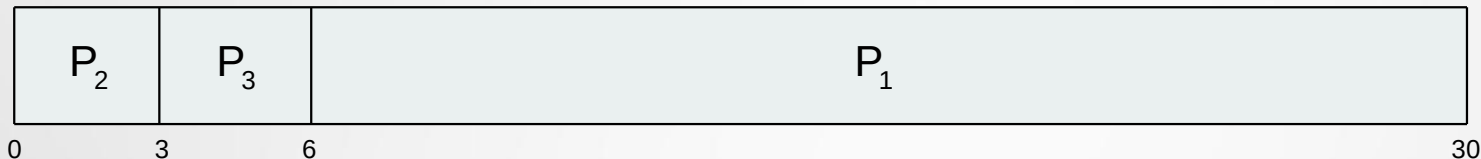


FCFS Scheduling (Cont.)

Suppose that the processes arrive in the order:

$$P_2, P_3, P_1$$

- The Gantt chart for the schedule is:



Waiting time		
P_1	P_2	P_3
6	0	3

Turnaround time		
P_1	P_2	P_3
30	3	6

Response time		
P_1	P_2	P_3
6	0	3

- Waiting time for $P_1 = 6$; $P_2 = 0$; $P_3 = 3$
- Average waiting time: $(6 + 0 + 3)/3 = 3$
- Average Turnaround time: $(30 + 3 + 6)/3 = 13$
- Average Response time: $(6 + 0 + 3)/3 = 3$
- Much better than previous case

Convoy effect - short process behind long process

- Consider one CPU-bound and many I/O-bound processes

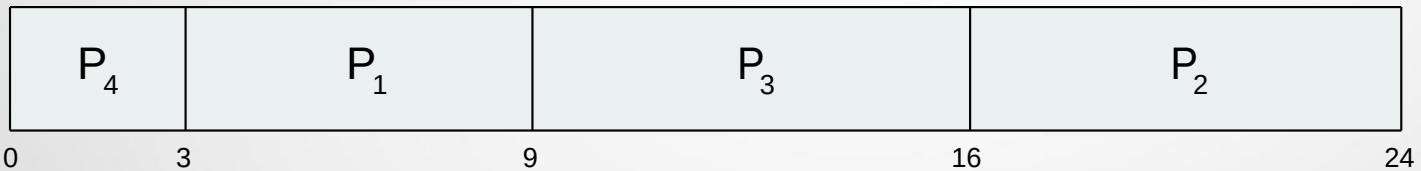
Shortest-Job-First (SJF) Scheduling

- Associate with each process the length of its next CPU burst
 - Use these lengths to schedule the process with the shortest time
- SJF is optimal – gives minimum average waiting time for a given set of processes
 - The difficulty is knowing the length of the next CPU request
 - Could ask the user

Example of SJF

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
P_1	0.0	6
P_2	2.0	8
P_3	4.0	7
P_4	5.0	3

- SJF scheduling chart



Waiting time			
P_1	P_2	P_3	P_4
3	16	9	0

Turnaround time			
P_1	P_2	P_3	P_4
9	24	16	3

Response time			
P_1	P_2	P_3	P_4
3	16	9	0

- Average waiting time = $(3 + 16 + 9 + 0) / 4 = 7$
- Average Turnaround time = $(9 + 24 + 16 + 3) / 4 = 13$
- Average Turnaround time = $(3 + 16 + 9 + 0) / 4 = 7$

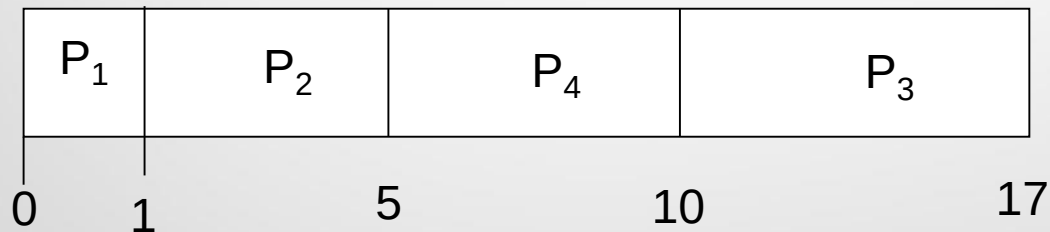
Example of Shortest-Job-first-time SJF

Non-Preemptive SJF

- Now we add the concepts of varying arrival times and preemption to the analysis

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
P_1	0	1
P_2	1	4
P_3	2	7
P_4	3	5

- Non-Preemptive SJF Gantt Chart*



- Average waiting time = $[(0-0)+(1-1)+(10-2)+5-3)]/4 = 26/4 = 2.5$ msec

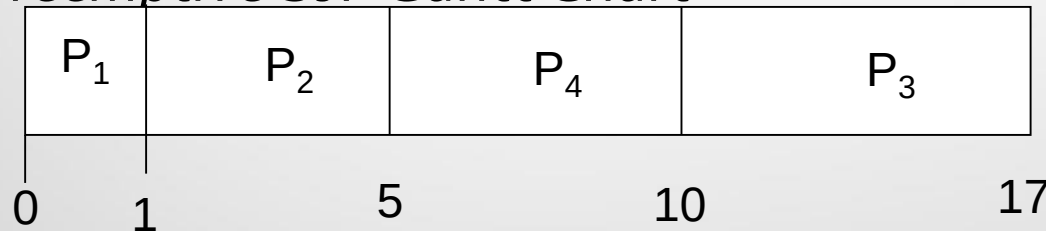
Example of Shortest-Job-first-time SJF

Non-Preemptive SJF

- Now we add the concepts of varying arrival times and preemption to the analysis

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
P_1	0	1
P_2	1	4
P_3	2	7
P_4	3	5

- Non-Preemptive SJF Gantt Chart*



Waiting time			
P_1	P_2	P_3	P_4
0-0	1-1	10-2	5-3

Turnaround time			
P_1	P_2	P_3	P_4
0-1= 1	1-5= 4	17- 2= 15	10- 3= 7

Response time			
P_1	P_2	P_3	P_4
0	0	10- 2= 8	5- 3= 2

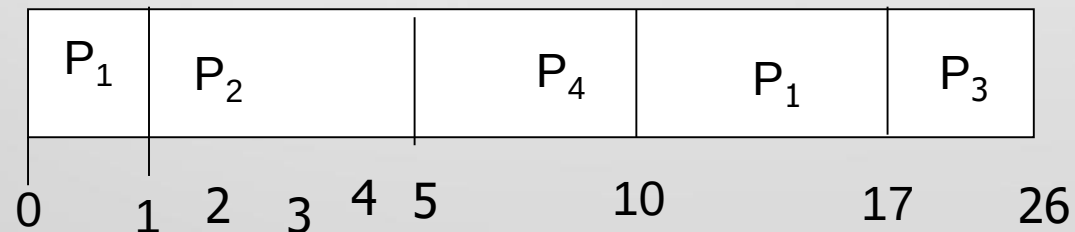
Example of Shortest-Job-first-time SJF

Preemptive SJF

- Now we add the concepts of varying arrival times and preemption to the analysis

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5

- Preemptive SJF Gantt Chart*



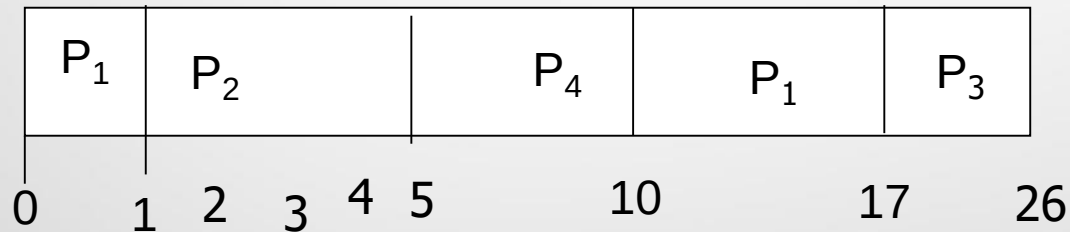
Example of Shortest-Job-first-time SJF

Preemptive SJF

- Now we add the concepts of varying arrival times and preemption to the analysis

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5

- Preemptive SJF** Gantt Chart



Waiting time			
P_1	P_2	P_3	P_4
10-1=9	1-1=0	17-2=15	5-3=2

Turnaround time			
P_1	P_2	P_3	P_4
17-0=17	5-1=4	26-2=24	10-3=7

Response time			
P_1	P_2	P_3	P_4
0-0=0	1-1=0	17-2=15	5-3=2



Thank you